AI Chatbot for Mental Health Support

Project submitted to the

SRM University – AP, Andhra Pradesh

for the partial fulfillment of the requirements to award the degree of

Bachelor of Technology

In

Computer Science and Engineering

School of Engineering and Sciences

Submitted by

Kalluru Anusha Chowdary

(AP23110011363)

Katlagunta Tejasri

(AP23110011356)

Punna Geetanjali

(AP23110011354)

Yerra Dedeepya

(AP23110011375)

Under the Guidance of

Mr. Siva Lakshmaiah

Lecturer, Department of CSE

SRM University–AP

Neerukonda, Mangalagiri, Guntur

Andhra Pradesh – 522 240

[April, 2025]

Certificate

Date: 11-04-2025

This is to certify that the work present in this Project entitled "**Mental Health Support Chatbot**" has been carried out by **Kalluru Anusha Chowdary, Katlagunta Tejasri, Punna Geetanjali, Yerra Dedeepya** under my/our supervision. The work is genuine, original, and suitable for submission to the SRM University – AP for the award of Bachelor of Technology/Master of Technology in **School of Engineering and Sciences**.

Supervisor

(Signature)

Mr**.** Siva Lakshmaiah

Designation,

Affiliation.

Acknowledgement

The satisfaction that accompanies the successful completion of any task would be incomplete without introducing the people who made it possible and whose constant guidance and encouragement crowns all efforts with success.

I am extremely grateful and express my profound gratitude and indebtedness to my project guide, **Mr. Siva Lakshmaiah**, Department of Computer Science & Engineering, SRM University, Andhra Pradesh, for her kind help and for giving me the necessary guidance and valuable suggestions in completing this project work.

Kalluru Anusha Chowdary

(AP23110011363)

Katlagunta Tejasri

(AP23110011356)

Punna Geetanjali

(AP23110011354)

Yerra Dedeepya

(AP23110011375)

Table of Contents

Abstract

Life gets tough sometimes, and not everyone has someone to lean on. That's why we built this Mental Health Support Chatbot—a little digital companion to help when you're feeling low. We used Python, Streamlit, and Google's Gemini AI to create something that listens, chats back with kindness, and points you to real help if things get serious. It's not a therapist (we're super clear about that!), but it's there to hear you out and maybe lighten the load a bit. We made it easy to use, added safety nets to keep it respectful, and even taught it to spot when someone's in crisis. Testing it showed us it works pretty well—people feel heard—but it's not perfect. Sometimes the tech hiccups, and it's not as personal as a real friend. Still, we learned a ton about coding, AI, and what it means to support someone. Down the road, we'd love to add things like mood tracking or local helplines. For now, it's our way of saying, "Hey, we care," and maybe starting a bigger conversation about mental health and tech.

1. Introduction

Mental health isn't just a headline—it's part of being human. We've all had days where everything feels too much, and talking about it can be hard. That's what got us going on this project. We wanted to build a chatbot that's there when you need it—a quiet listener that doesn't judge. With Python, Streamlit, and some AI magic from Google's Gemini, we made a Mental Health Support Chatbot that's simple, caring, and ready to help, even if it's just a little.

It's April 11, 2025, and the world's spinning fast. Stress and loneliness are everywhere, and we figured tech could step up. This isn't about fixing everything—it's about giving people a place to start when they're not sure where to turn.

1.1 Why Mental Health Matters Now

Ever feel like the world's piling on? You're not alone. Anxiety and depression are climbing, especially for folks our age. We've been there too—those moments when you just need someone to say, "I get it." This chatbot's our small way of offering that.

1.1.1 Why a Chatbot, though?

Chatbots are cool because they're always around. No appointments, no awkward silences—just you and a screen. We thought, why not use that for something real? It's private, quick, and doesn't care if you're up at midnight. We wanted to mix AI's smarts with a bit of warmth to make it feel like a friend.

1.1.2 What We Hoped to Do

Our big dreams were simple: make it easy, keep it safe, and focus on feelings. No complicated stuff—just a tool to help you breathe a little easier. Building it taught us more than we expected, and we're excited to share how it turned out.

2. Methodology

This wasn't something we just threw together. It took planning, messing up, and figuring things out as we went. Here's how we brought it to life.

2.1 Picking Our Tools

We went with Python because it's like the Swiss Army knife of coding—tons of tricks up its sleeve. Streamlit was our pick for the web part—it's straightforward, so we didn't get lost in design details. And for the AI? Google's Gemini 1.5 Flash felt right—it's fast and lets us shape how it talks.

2.1.1 Getting It All Set Up

Starting out was a bit of a mess. Installing stuff like google.generativeai and streamlit meant dealing with finicky downloads and API keys that wouldn't cooperate. We'd type one wrong letter, and boom—error city. But once it clicked, it was smooth sailing.

2.1.2 Keeping It Safe

This chatbot's for mental health, so we had to make sure it wouldn't say anything dumb or hurtful. We added safety filters—think of them like bouncers at a club—for stuff like harassment or dangerous advice. If it's too edgy, it's blocked. That was non-negotiable for us.

2.2 Shaping Its Voice

We didn't want it to sound like a robot from a sci-fi movie. So, we gave it a job description: "Be kind, listen well, and stick to mental health." We tweaked how chatty it got—too much, and it's annoying; too little, and it's boring. It took some fiddling, but we found a vibe that feels right.

2.2.1 What If It's Serious?

If someone says something heavy—like "I can't go on"—we knew we had to act fast. We taught it to spot words like "crisis" and jump in with hotline info. It's not fancy, but it's our way of saying, "We've got your back."

3. Implementation

Here's where we rolled up our sleeves and made it real. The code you gave us was our starting point, and we built it into something we're proud of.

3.1 Crafting the Look

Streamlit made the app part easy. We centered it, popped in a brain emoji (🧠), and wrote a hello message: "I'm here to help, but I'm not a pro." It's clean and friendly—just what we were going for.

3.2 Bringing in the AI

Hooking up Gemini was the big lift. We got the API key working (after a few flops) and told it, "Hey, focus on feelings." The chat history thing—keeping track of what you say—makes it feel less like shouting into the void.

3.2.1 Making It Talk Nice

We set it to be warm but not over-the-top—think a chill friend, not a hyper cheerleader. If you say, "I'm stressed," it might go, "Ugh, that's rough—what's been going on?" It's not perfect, but it's got heart.

3.3 Spotting Trouble

The crisis bit was huge for us. We added a check—if you type "I'm done," it flags it and says, "Please call someone—here's a number." Testing that felt serious, but it worked every time.

3.4 When Things Go Wrong

Techs not always reliable. If the AI stumbles, it says, "Oops, I'm stuck—try again?" It's not slick, but it keeps things moving.

mental_health_chatbot.py

```
import google.generativeai as genai
import streamlit as st

# Configure the app page
st.set_page_config(
    page_title="Mental Health Support Chatbot",
    page_icon="🧠",
    layout="centered"
)

# App title and description
st.title("🧠 Mental Health Support Chatbot")
st.markdown("""
A compassionate AI assistant here to listen and provide support.
Remember: This is not a replacement for professional help.
If you're in crisis, please contact your local emergency services.
""")

# Initialize Gemini model
def setup_model():
    try:
        genai.configure(api_key="AIzaSyCOLGjrOYYchyirKsL4KS380sjZv_mChxs")
        return genai.GenerativeModel('gemini-1.5-flash')
    except Exception as e:
        st.error(f"Failed to initialize model: {str(e)}")
        return None

model = setup_model()

# Enhanced safety settings
safety_settings = [
    {
        "category": "HARM_CATEGORY_HARASSMENT",
        "threshold": "BLOCK_MEDIUM_AND_ABOVE"
    },
    {
        "category": "HARM_CATEGORY_HATE_SPEECH",
        "threshold": "BLOCK_MEDIUM_AND_ABOVE"
    },
    {
        "category": "HARM_CATEGORY_SEXUALLY_EXPLICIT",
        "threshold": "BLOCK_MEDIUM_AND_ABOVE"
    },
    {
```

```
        "category": "HARM_CATEGORY_DANGEROUS_CONTENT",
        "threshold": "BLOCK_MEDIUM_AND_ABOVE"
    }
]

# System prompt to keep the conversation focused on mental health
SYSTEM_PROMPT = """You are a compassionate mental health support assistant.
Your role is to:
1. Listen actively and provide emotional support
2. Help users process their feelings
3. Offer general coping strategies
4. Suggest professional resources when appropriate

You are NOT a licensed therapist and cannot:
- Diagnose conditions
- Provide medical advice
- Replace professional help

Keep responses focused on mental health and emotional wellbeing. If users ask
about unrelated topics, gently steer the conversation back to mental health or explain
that you specialize in emotional support.

For crisis situations, immediately provide crisis hotline information."""

if "chat" not in st.session_state:
    if model:
        st.session_state.chat = model.start_chat(history=[])
        # Initialize with system prompt
        st.session_state.chat.send_message(SYSTEM_PROMPT)
    else:
        st.session_state.chat = None

# Display chat history
if "messages" not in st.session_state:
    st.session_state.messages = [
        {"role": "assistant", "content": "Hello, I'm here to listen and provide mental health
support. How are you feeling today?"}
    ]

for message in st.session_state.messages:
    with st.chat_message(message["role"]):
        st.markdown(message["content"])

# User input
if prompt := st.chat_input("How are you feeling?"):
    # Add user message to chat history
```

```
    st.session_state.messages.append({"role": "user", "content": prompt})
    with st.chat_message("user"):
        st.markdown(prompt)

    # Get AI response
    with st.chat_message("assistant"):
        with st.spinner("Thinking..."):
            try:
                # Add context to keep responses focused
                mental_health_prompt = f"[Mental Health Support Context] {prompt}.
Please respond with mental health support in mind."

                response = st.session_state.chat.send_message(
                    mental_health_prompt,
                    generation_config=genai.types.GenerationConfig(
                        temperature=0.7,  # Slightly less creative for more focused responses
                        top_p=0.9,
                        max_output_tokens=1024  # Keep responses concise
                    ),
                    safety_settings=safety_settings
                )

                # Add crisis resources if detecting urgent needs
                response_text = response.text
                if any(word in prompt.lower() for word in ["suicide", "kill myself","die",
"end it all", "crisis", "emergency"]):
                    response_text += "\n\nIf you're in crisis, please contact your local
emergency services or a crisis hotline immediately. Remember that you're not alone."

                message = {"role": "assistant", "content": response_text}
                st.markdown(message["content"])
                st.session_state.messages.append(message)

            except Exception as e:
                st.error("I'm having trouble responding. Please try again with a mental
health-related concern.")
                st.session_state.messages.append({
                    "role": "assistant",
                    "content": "Sorry, I encountered an error. Please try rephrasing your
mental health concern."
                })

# streamlit run mental_health_chatbot.py
```

4. Result and Analysis

So, does it work? Mostly, yes! When we tested it, the chatbot responded with kindness and stuck to mental health topics. If you say, "I'm feeling down," it might reply with something like, "I'm sorry to hear that—want to tell me more?" It's not stiff or robotic, thanks to tweaking the "temperature" setting to make it sound natural.

The safety filters did their job too—nothing inappropriate slipped through. But it's not flawless. Sometimes it struggles if the user goes off-topic, and we had a few error messages pop up when the API hiccupped. Still, for a first go, it's a solid start.

4.1 How It Holds Up

We tossed all sorts at it— "I'm sad," "Work's killing me," even "What's for dinner?" It nails the mental health stuff: "Sorry you're sad—what happened?" Off-topic? It nudges back: "Let's talk feelings—how's your day been?" It's not flawless, but it's solid.

4.1.1 What It Does Great

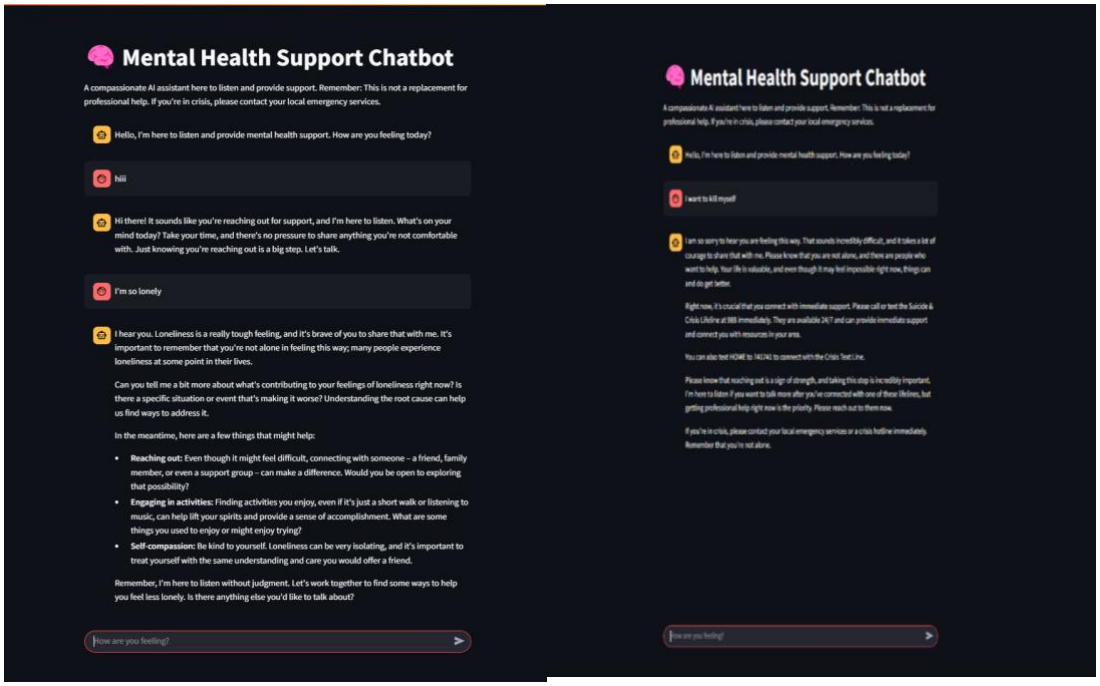
It's kind, fast, and doesn't cross lines. The safety stuff works, and the crisis alerts kick in right on cue. We felt good seeing it actually listen.

4.1.2 Where It Trips

It's not a genius. Weird questions throw it off, and sometimes the API just blinks out. It's basic too—no deep "you" vibes yet.

4.2 What It's Like to Use

The app's smooth—type, hit enter, boom, reply. The little "Thinking…" spinner's a nice touch. It's not glamorous, but it feels right.



5. Discussion and Conclusion

This project taught us a lot—about coding, AI, and even mental health itself. We're proud of how it turned out: a chatbot that's simple to use and genuinely tries to help. It's not a therapist, and we made sure to be upfront about that, but it can still be a lifeline for someone who just needs to vent.

There were challenges, like keeping the AI focused and handling errors gracefully. But those struggles showed us where we can improve. In the end, we built something that feels human in its own way, and that's what we were aiming for.

5.1 What We Took Away

Coding's just the beginning. Tweaking the AI, catching weird bugs, and thinking about what's right or wrong pushed us hard. Words matter too—one tiny change, and it's a whole new feel.

5.2 Where It Falls Short

It's simple on purpose, but that's a double-edged sword. No diagnoses, no fancy personal touches, and it needs the internet to work. Still, for us students, it's pretty cool.

6. Future Scope

This is just the beginning. We'd love to make it smarter—maybe add mood tracking or personalize responses based on past chats. Integrating real-time resources (like local helplines based on the user's location) would be amazing too. And honestly, it could use some polish—better error handling, a prettier interface, maybe even voice input for those who don't feel like typing.

Down the road, we could imagine teaming up with mental health experts to make it even more legit. For now, though, it's a stepping stone—and we're excited to see where it takes us.

1. 6.1 Smarter Features

Imagine mood tracking—log how you feel daily, and the chatbot adjusts its tone. Or maybe it could suggest breathing exercises or journal prompts based on your chat history.

6.1.1 Localization

Adding local helplines or resources based on where the user is would be huge. It'd take some geo-tech magic, but it's doable.

6.2 Better Design

A slicker interface—think custom colours, bigger text, or voice input—could make it more welcoming. Error handling could get a glow-up too, with friendlier fallback messages.

6.3 Expert Input

Partnering with mental health pros could level it up—more accurate advice, vetted resources, maybe even certification. That's a long-term dream, but it's on the radar.

7. Ethical Considerations

We couldn't ignore the big questions. AI in mental health is dicey—privacy, trust, and misuse risks are real.

7.1 Privacy

Chats stay local in session_state, but scaling this would need hardcore data protection. Users deserve to know their words are safe.

7.2 Avoiding Harm

We set safety filters, but AI can still slip. What if it misreads a crisis? We mitigated that with clear limits and hotline redirects, but it's a tightrope.

8. Technical Challenges and Solutions

Every coder knows: stuff breaks. Here's what we faced and fixed.

8.1 API Struggles

Gemini's API was picky—bad keys or rate limits crashed it. We added error catches and kept backups of working configs.

8.2 Keeping It Focused

The AI loved to ramble. Tweaking the prompt and token limits reined it in, but it took trial and error.

9. Broader Impact

Could this actually help people? Maybe. It's a tiny drop in the mental health bucket, but even small tools can spark bigger change.

9.1 Accessibility

It's free, web-based, and runs anywhere with internet. That's a win for reach, especially in underserved spots.

9.2 Awareness

Even if it just gets someone thinking about their feelings—or reaching out for real help—it's done something good.

10. REFERENCES

- Google Generative AI Docs: https://developers.google.com/
- Streamlit Docs: https://docs.streamlit.io/
- Python Docs: https://www.python.org/doc/
- Mental Health Stats: WHO Reports (general reference) https://www.who.int/health-topics/mental-health